

UNIT I – LM 4

Docker Containers

1.1 The Evolution of Deployment

In traditional computing, applications were installed directly on physical servers, leading to resource underutilization and "dependency hell." Cloud computing transformed this by providing instant access to virtualized resources over the internet

While Virtual Machines (VMs) were the initial standard, Containerization has emerged as a more agile, lightweight alternative

1.2 Defining Docker

- Docker is an open-source platform that enables developers to separate applications from their infrastructure to deliver software quickly
- It uses operating system-level virtualization to deliver software in packages called containers
- These containers bundle the application code with its specific environment, including libraries, system tools, and runtimes

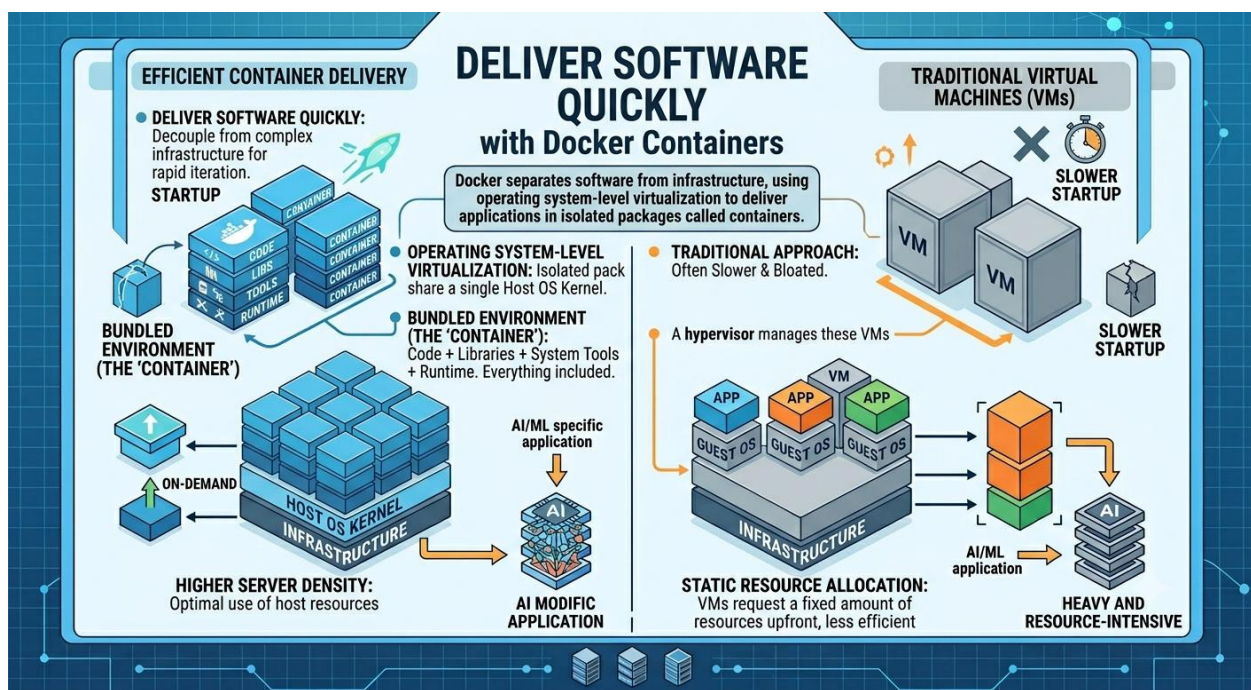


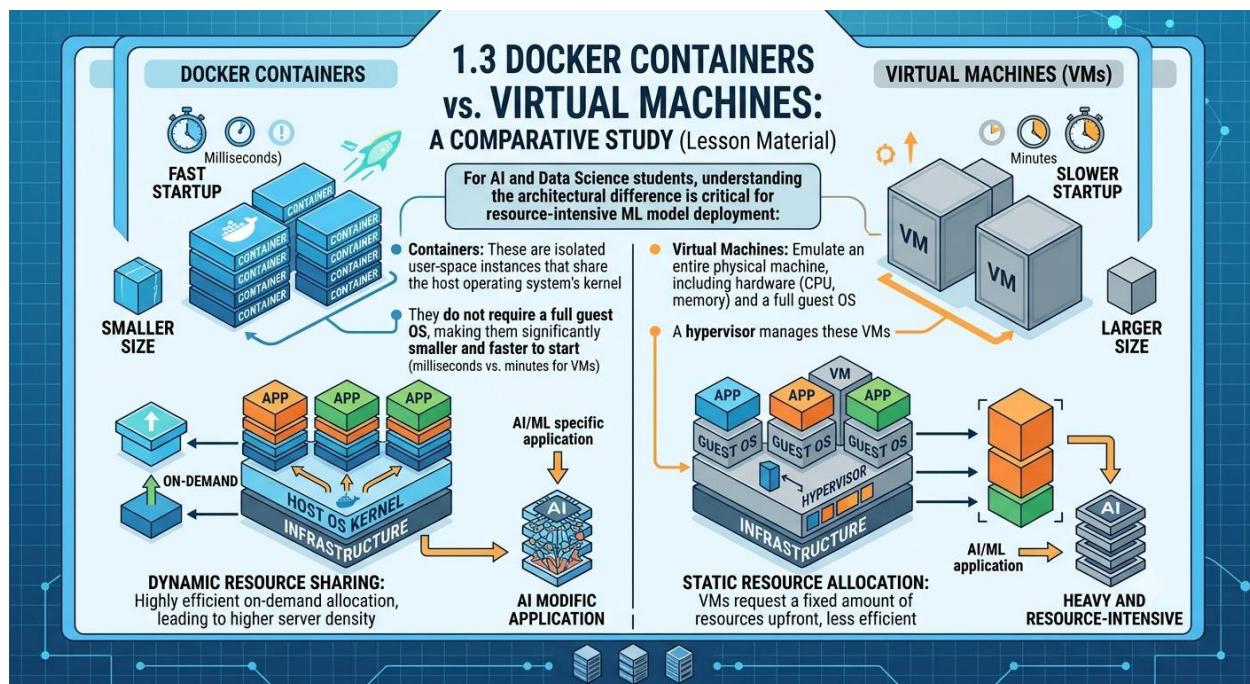
Fig: Docker

1.3 Docker vs. Virtual Machines (A Comparative Study)

For AI and Data Science students, understanding the architectural difference is critical for resource-intensive ML model deployment:

Virtual Machines: Emulate an entire physical machine, including hardware (CPU, memory) and a full guest OS

A hypervisor manages these VMs



Docker Containers: These are isolated user-space instances that share the host operating system's kernel

They do not require a full guest OS, making them significantly smaller and faster to start (milliseconds vs. minutes for VMs)

Resource Sharing: VMs request a fixed amount of resources upfront, while Docker containers use resources on-demand from the kernel, leading to higher server density

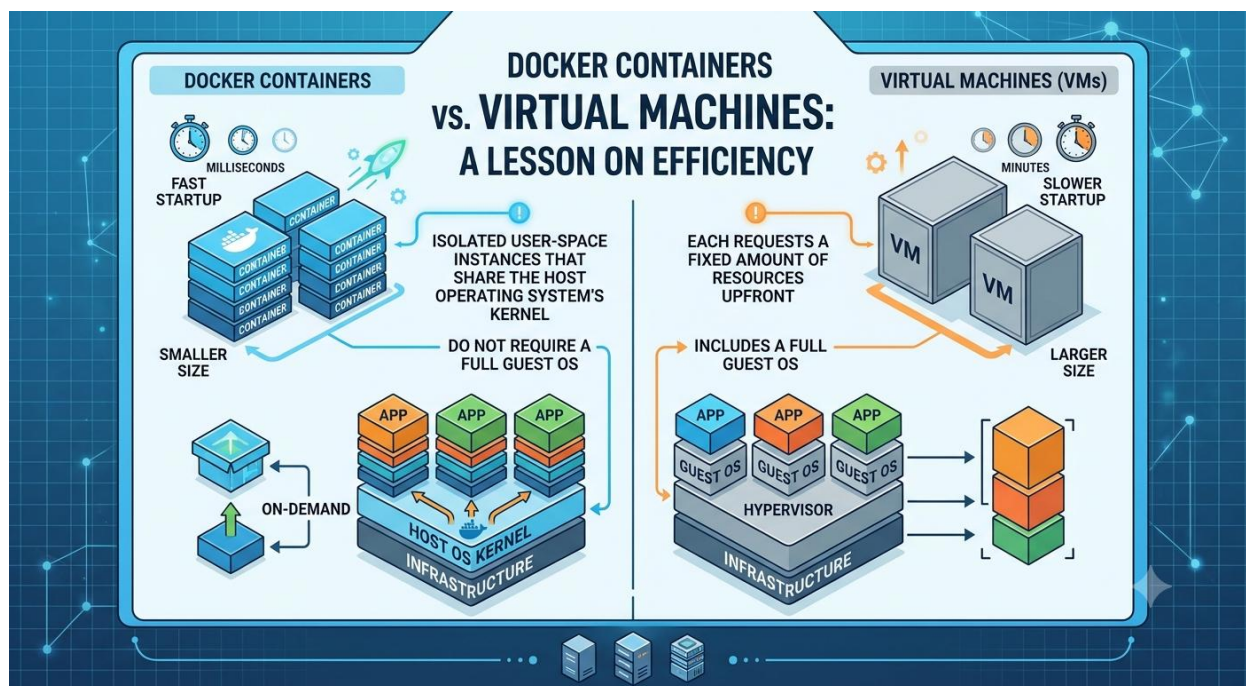


Fig: Docker Containers Vs Virtual Machines

2.1 The Client-Server Model

Docker uses a client-server architecture:

- **Docker Daemon (dockerd):** A persistent background process that manages Docker objects like images, containers, networks, and volumes
- **Docker Client (docker):** The primary CLI tool users use to interact with the daemon (e.g., executing `docker run`)
- **Docker API:** The interface used by the client to communicate with the daemon over UNIX sockets or network interfaces

2.2 Essential Docker Objects

Images: Read-only templates used to build containers

- They are built in layers; if a layer changes, only that layer is rebuilt, making images lightweight

Containers:

- The runnable instances of an image
- They are immutable entities that should be destroyed and replaced rather than updated in place
- Registries: Repositories for storing and sharing images. Docker Hub is the default public registry
- Volumes: Mechanisms to facilitate independent persistence of data, ensuring data remains even after a container is deleted

Technical Implementation

3.1 Linux Kernel Primitives

Docker relies on specific Linux kernel features to achieve isolation:

Namespaces: Provide isolation for process trees, network stacks, user IDs, and mounted file systems

This ensures a container only sees the resources allocated to it

Control Groups (cgroups): Used for resource allocation, limiting how much CPU, memory, or disk I/O a container can consume

Union File Systems (e.g., OverlayFS): Allows containers to run within a single Linux instance by layering file systems, avoiding the overhead of VMs

3.2 The Dockerfile

A Dockerfile is a text file containing the declarative instructions to build an image

Example Instruction: Specifying the Linux distribution, installing Python for AI libraries, and copying the application source code

Docker in AI and Data Science Workflows

Solving "Works on My Machine"

In Data Science, environment consistency is vital. A model trained on a laptop must behave identically on a cloud GPU cluster. Docker packages the entire user-space requirement (runtime, third-party libraries, init scripts) into an immutable snapshot

High-Performance Compute and GPU Support

By 2026, AI-native architectures have made containers the backbone for training and inference

Containers vs. Serverless for AI: While serverless (like AWS Lambda) is excellent for ingest, containers thrive in training and inference because serverless generally lacks native GPU support

Workload Placement: AI pipelines often use hybrid models—serverless for triggers and containerized GPU clusters for heavy compute

Managed Cloud Container Services:

When moving Docker to the cloud, providers offer varying levels of management:

Amazon Web Services (AWS) Options

- Amazon ECS (Elastic Container Service): A proprietary, simple-to-use orchestration service deeply integrated with AWS
- Amazon EKS (Elastic Kubernetes Service): Brings open-source Kubernetes to AWS, offering high portability and fine-grained control for complex microservices
- AWS Fargate: A serverless compute engine for containers. It removes the need to manage underlying EC2 instances, allowing users to run containers directly

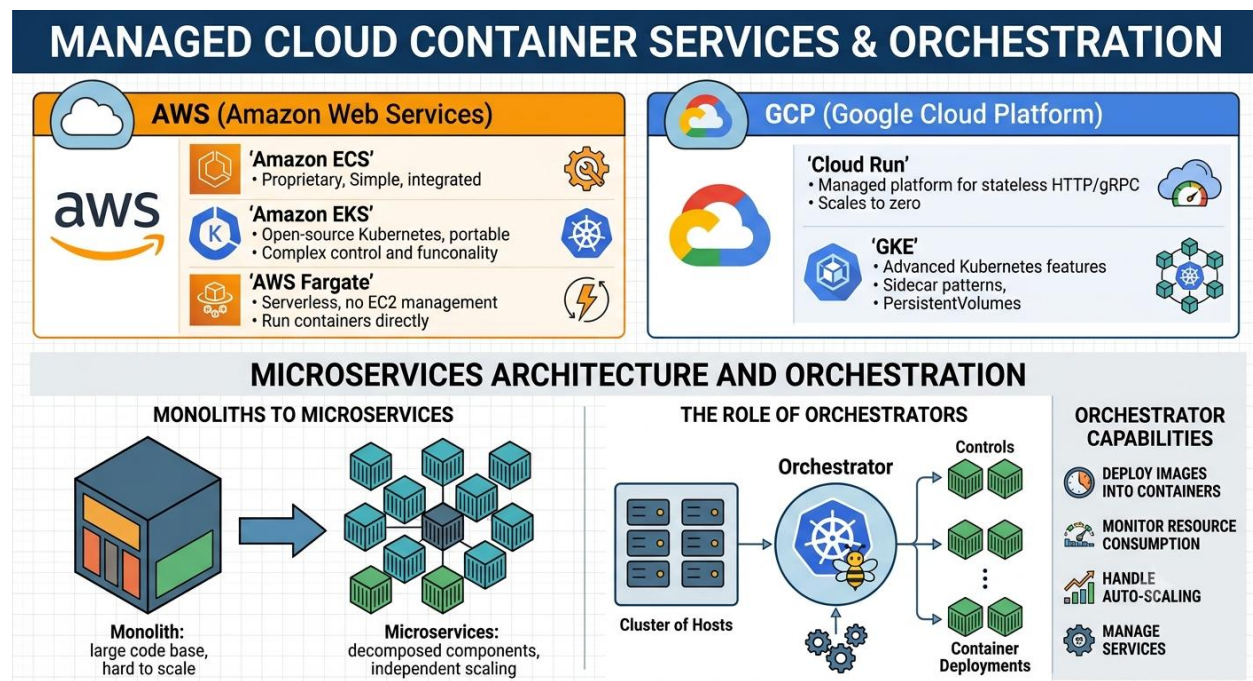


Fig: Cloud Container Service & Orchestration

Google Cloud Platform (GCP) Options:

Cloud Run: A managed platform for stateless HTTP/gRPC services that scales to zero

GKE (Google Kubernetes Engine): The choice for workloads needing advanced Kubernetes features like sidecar patterns or PersistentVolumes

Microservices Architecture and Orchestration

From Monoliths to Microservices

Microservices decompose an application into many small components, each running in its own container

This increases flexibility and scalability but introduces the challenge of managing thousands of objects

The Role of Orchestrators

Orchestrators (e.g., Kubernetes, Docker Swarm) manage the deployment of images into containers across clusters of hosts

Capabilities: They monitor resource consumption, handle auto-scaling, manage service discovery, and provide overlay networks to isolate communication between apps

Cost Optimization Strategies

For cloud-based AI projects, cost management is a primary concern

Rightsizing and Scaling

Rightsizing: Selecting the correct instance type and size based on actual workload needs to prevent overprovisioning

Auto-Scaling: Automatically adjusting computing resources based on real-time traffic to reduce idle costs

Pricing Models

Spot Instances: Using spare AWS capacity at up to 90% discount, ideal for fault-tolerant AI training jobs

Reserved Instances: Long-term commitments for predictable, steady-state workloads

Serverless Containers

- Using Fargate or Cloud Run eliminates costs for idle resources since you pay only for the time the container is executing
- Security in Containerized Environments (NIST SP 800-190)

Container security must be addressed across five architectural tiers:

The Five Tiers of Security

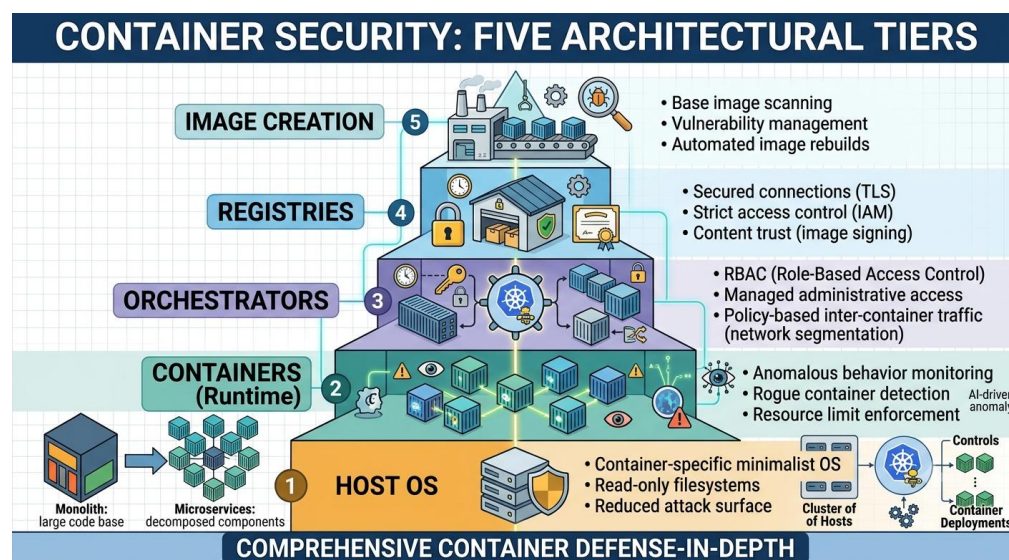
Image Creation: Vulnerability management for base layers.

Registries: Securing connections and implementing access control

Orchestrators: Managing administrative access and inter-container traffic

Containers (Runtime): Monitoring for anomalies and rogue containers

Host OS: Using container-specific host OSs (minimalist, read-only) to reduce attack surfaces



Key Countermeasures

- **Least Privilege:** Configure containers to run as non-privileged users
- **Secret Management:** Never embed secrets (API keys, DB passwords) in images; inject them at runtime using orchestrator tools
- **Hardware Root of Trust:** Use Trusted Platform Modules (TPM) to provide a hardware-based chain of trust from boot to container runtime

Networking and Data Persistence

Container Networking

- Containers use Software-Defined Networking (SDN) and overlay networks to communicate privately
- **Challenge:** Traditional network tools are often "blind" to virtualized, encrypted inter-container traffic
- **Solution:** Use app-aware network filtering that dynamically generates rules based on container lifecycle

Persistent Storage

Since containers are stateless, AI datasets must be stored in external persistent data stores (e.g., Amazon S3, databases) to ensure data remains available when containers are updated.

Future Trends and 2026 Context

Hybrid Architectures

By 2026, over 78% of engineering teams run hybrid models. There is no single "winner" between serverless and containers; instead, smart teams optimize both for cost and performance.

Sustainability and Edge Computing:

Edge Containers: CDNs now run containerized functions near users with <10ms latency.

Green Cloud: Pay-as-you-use models (like serverless containers) align with sustainability mandates to reduce wasted idle compute.

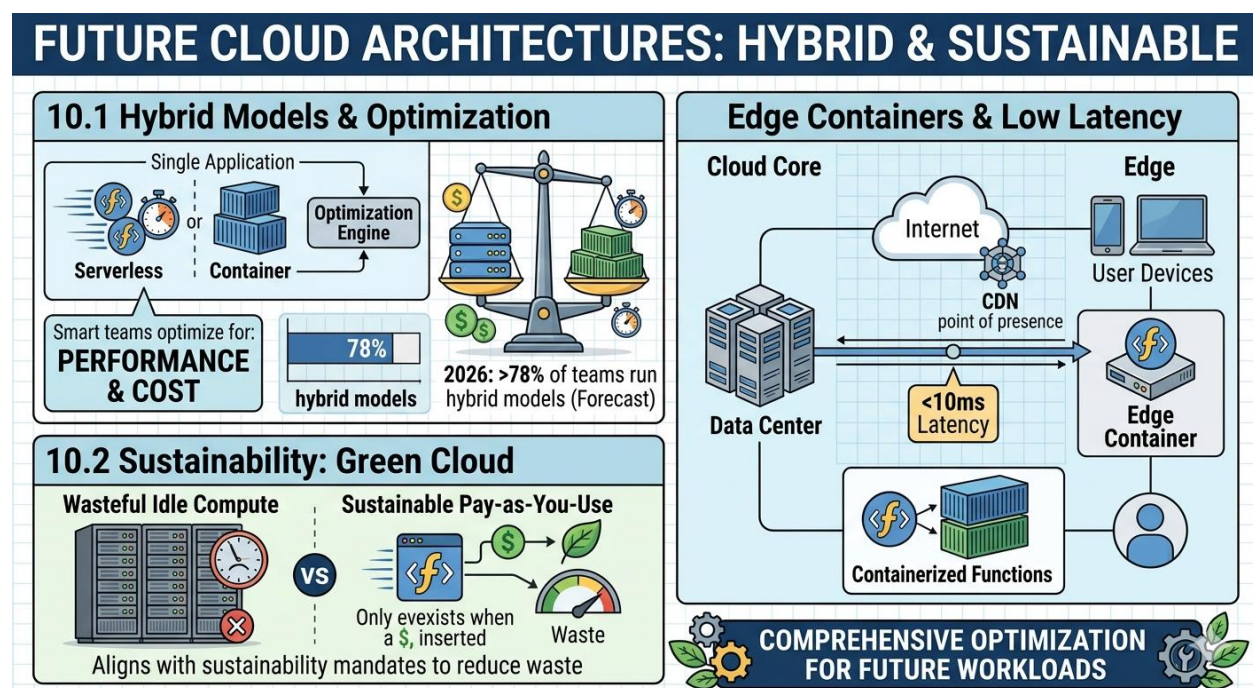


Fig: Future Cloud Architecture

Docker containers are widely used across the tech industry because they solve the classic "it works on my machine" problem by packaging code with all of its dependencies. In 2026, Docker serves as a foundational tool for cloud-native infrastructure, DevOps, and advanced data workflows.

The primary real-world applications for Docker containers include:

1. Microservices Architecture

Instead of building a massive, single-piece application (a monolith), modern teams break software down into small, independent components.

- **Application:** Each service (e.g., user authentication, payment processing, notification engine) runs inside its own isolated Docker container.

- **Benefit:** Teams can update, scale, or rewrite individual microservices without risking a crash of the entire application.

2. Continuous Integration & Continuous Deployment (CI/CD)

Docker is a cornerstone of automated testing and deployment pipelines.

- **Application:** When a developer pushes code, a CI/CD tool (like GitHub Actions or Jenkins) spins up a Docker container to compile, test, and validate the code in an exact replica of the production environment.
- **Benefit:** Eliminates environment mismatches and allows teams to ship updates multiple times a day with high confidence.

3. AI, Machine Learning, and Big Data

Managing local dependencies for data science (such as matching specific Python versions with precise CUDA and PyTorch libraries) is notoriously difficult.

- **Application:** Data scientists containerize data workflows, automated ETL processes, and ML models.
- **Benefit:** Guarantees absolute reproducibility of data pipelines and machine learning model training across different clusters and cloud GPU instances.

4. Cloud Migration and Hybrid Cloud Deployments

Organizations frequently need to shift legacy software off physical hardware and into modern infrastructure.

- **Application:** Legacy applications are packaged into Docker containers, making them "cloud-agnostic."
- **Benefit:** The software can seamlessly migrate between on-premises servers and public cloud providers (like AWS, Google Cloud, or Azure) without needing a code rewrite.

5. Local Development and Rapid Onboarding

Setting up a new developer's laptop with databases, message brokers, and correct runtimes can take days.

- **Application:** Using a configuration file called `docker-compose.yml`, a complex, multi-container stack can be spun up locally with a single command.
- **Benefit:** New team members can start coding on day one, confident that their local system mirrors exactly how the app runs in production.